

INF01151 – Sistemas Operacionais II

Aula 02 - Introdução à programação
concorrente

Prof. Alberto Egon Schaeffer Filho

Programação multi-thread x distribuída x paralela

- Programação multi-thread
 - Duas ou mais **tarefas** trabalhando na resolução de um problema
 - Pressupõe o paradigma de **memória compartilhada** (*shared variables*)
- Programação distribuída
 - Programa concorrente executado sobre um sistema computacional que possui **memória distribuída**
 - Comunicação através de **troca de mensagens** (*message-passing*)
 - Processo em um computador pode acessar (remotamente) dados em outros computadores
- Programação paralela
 - Programa concorrente, executando sobre *memória compartilhada ou memória distribuída*, com o objetivo de **reduzir tempo** de processamento
 - Executando “fisicamente” ao mesmo tempo

Introdução

- Tarefa:
 - Parte de um **conjunto de ações** para realizar um trabalho ou solucionar um problema
 - **Abstração** independente de modelo e linguagem de programação e de plataforma (hardware + software)
- As tarefas são implementadas através de:
 - Processos
 - Threads
 - Problemas de sincronização acontecem tanto em processos quanto em threads

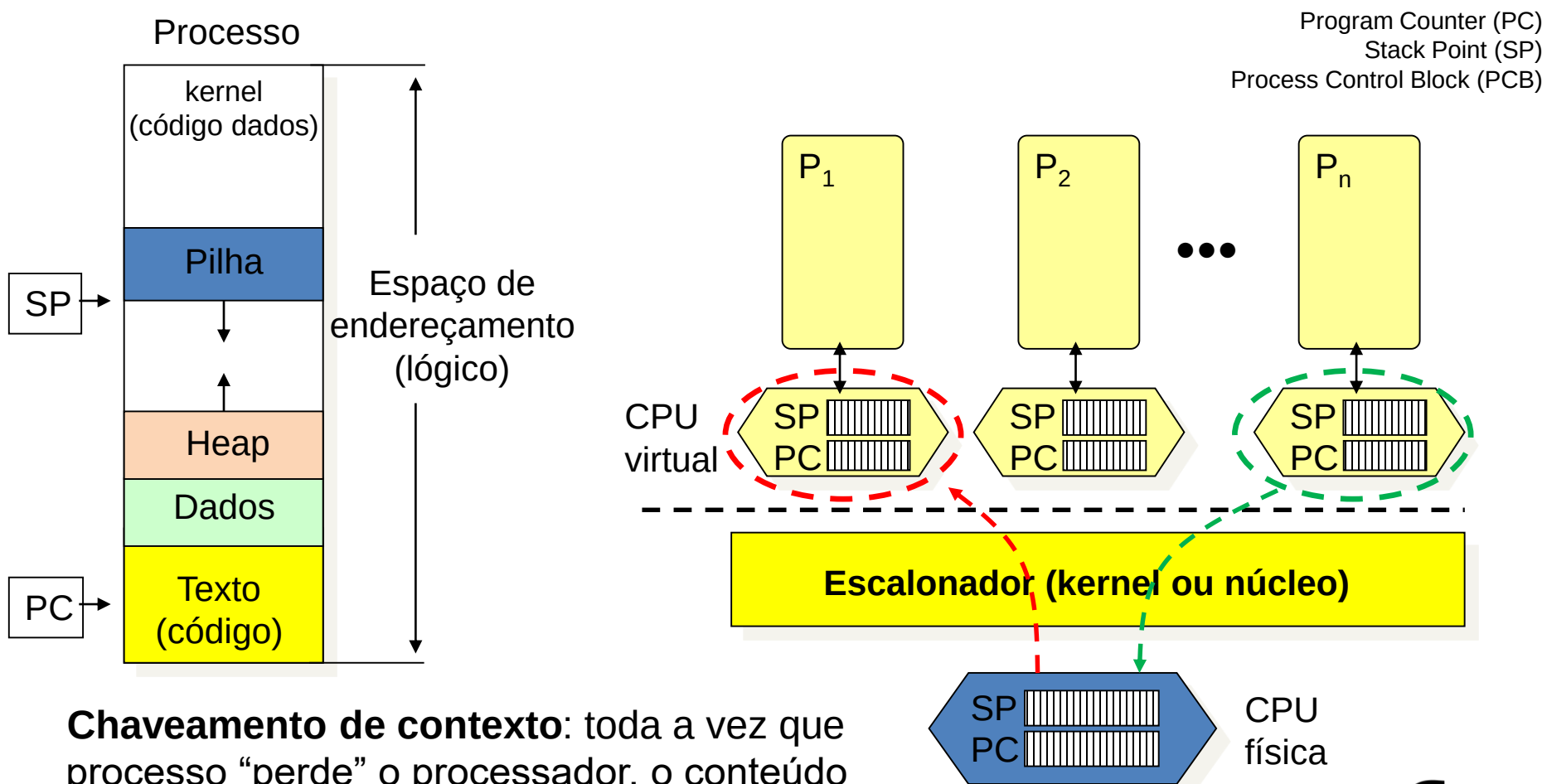
Criação de tarefas

- Questões de projeto: **Como? Por quem? Quando? Quantos?**
- Linguagens **imperativas** versus **declarativas**
 - **Imperativas**: criação é feita explicitamente pelo programador (como e quem)
 - Quando o programador quiser e quantos ele definir
 - **Declarativa**: a concorrência é implícita e partes do programa podem executar em paralelo, conforme determinado pelo compilador/runtime (como e quem)
 - Quando o compilador/runtime achar apropriado e quantos ele achar necessário
- Outro aspecto: **estático** versus **dinâmico**
 - **Estático**: programa declara um conjunto fixo de processos
 - **Dinâmico**: criados e terminados durante a execução por demanda

Representação de processo

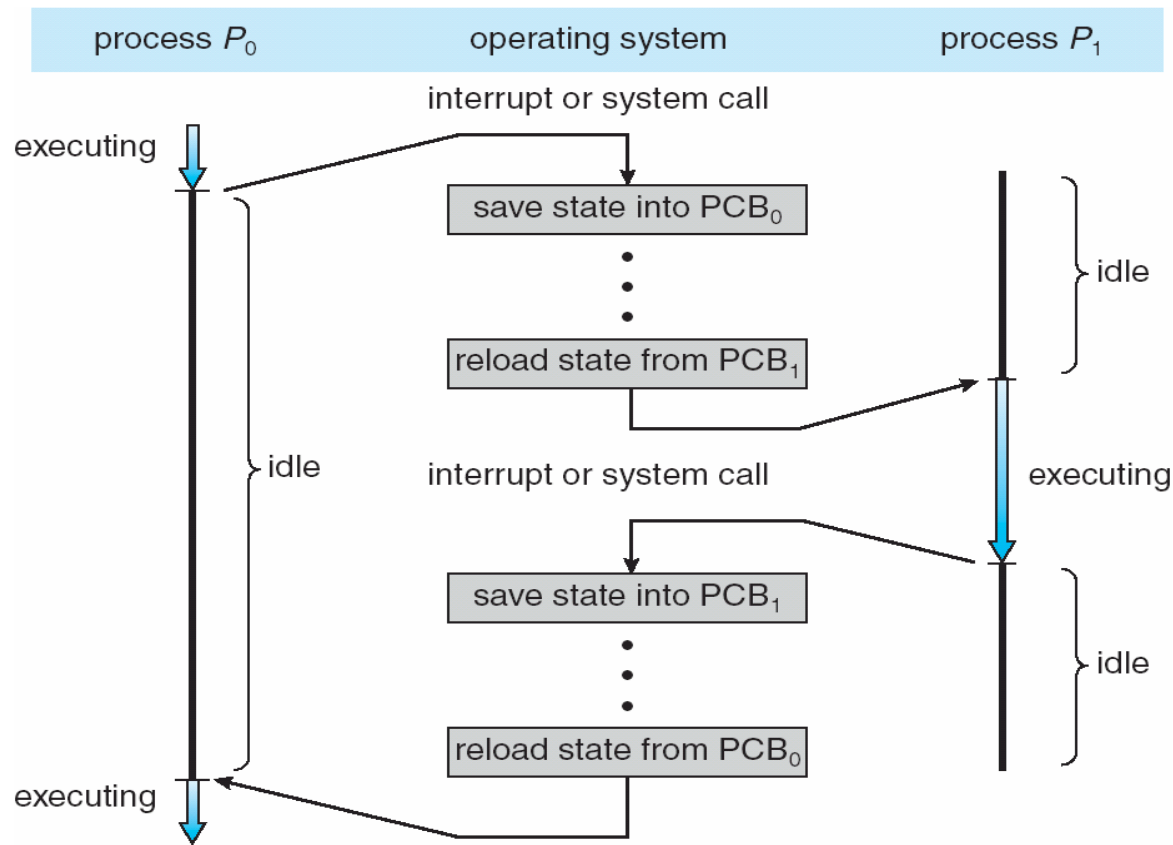
- Processo é um **programa em execução**
 - Programa é uma entidade *passiva*; processo é uma entidade *ativa*
 - Contém áreas na memória para código, dados e pilha
- Possui uma série de **estados (ready, running, blocked)** para representar sua evolução no tempo, implica:
 - Determinar eventos que realizam a transição entre os estados
 - Determinar quando um processo tem direito a “utilizar” o processador
 - Organizar processos nos diferentes estados em estruturas do kernel
 - E.g., **device** queue linked list, ready **queue** linked list, etc
- **Descritor específico** Process Control Block (PCB)
 - Usado para **manter informações** a respeito do processo
 - E.g., process state, program counter, CPU registers, CPU scheduling information, accounting information, memory-management information

A arte de enganar... processador como máquina virtual



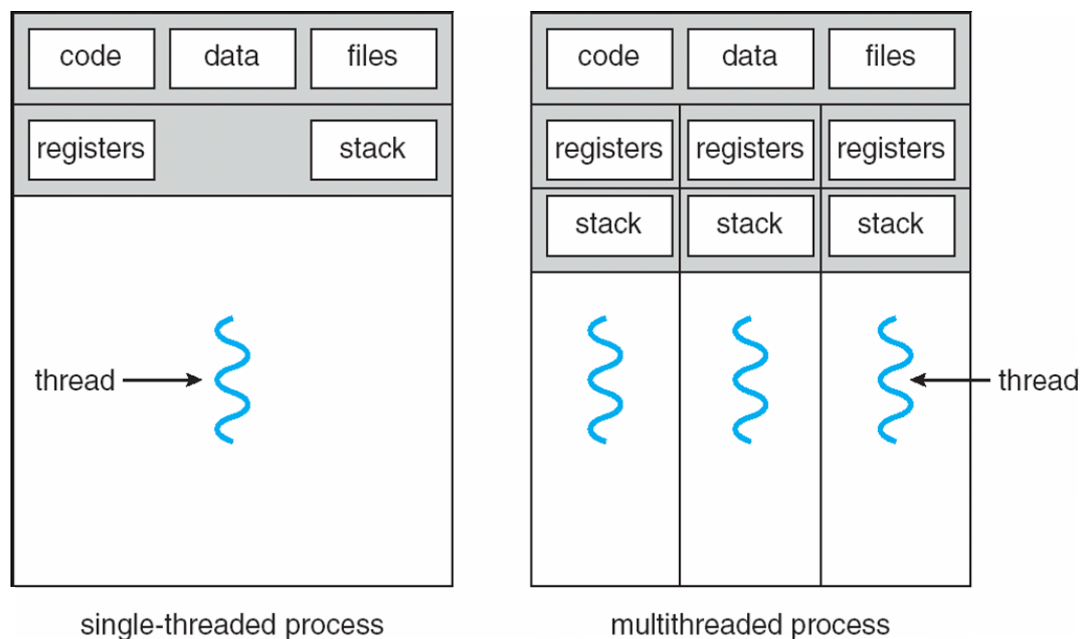
Chaveamento de contexto: toda a vez que processo “perde” o processador, o conteúdo dos registradores é salvo no seu descritor

CPU switch from process to process



Do modelo de processo para *multithreading*

- Thread:
 - Fluxo de execução independente dentro de um processo
 - Executam como parte de uma aplicação
 - Múltiplas partes da aplicação podem ser implementadas como threads separadas
 - Threads compartilham entre si os recursos de um processo



Cada thread contém:

- Thread ID
- Registradores (inc. PC)
- Stack

Compartilha com outras threads (mesmo processo)

- Code section
- Data section
- OS resources (files)

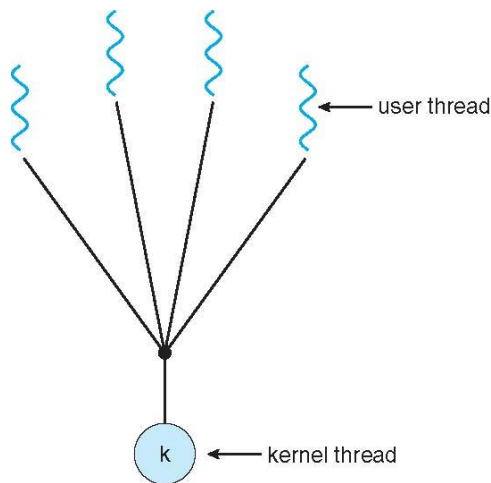
Recursos do sistema e as threads

- Gerência de threads é mais “leve”
 - Chaveamento de contexto entre duas threads de um mesmo processo é mais rápido, devido ao compartilhamento de recursos
- As threads dividem com os processos:
 - Espaço de endereçamento (código e dados)
 - Variáveis globais
 - Arquivos abertos, diretório corrente, sinais, processos filhos
 - Informações de contabilização, estatísticas do SO, registro descritor
- Porém, tem seus próprios recursos:
 - Pilha
 - Registradores, incluindo stack pointer (SP) e program counter (PC)

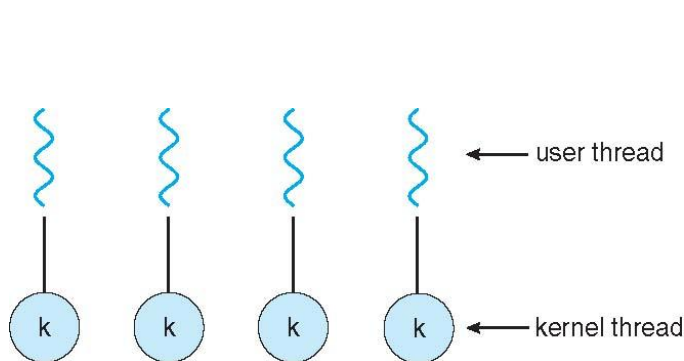
Modelos de suporte a multithreading

- Suporte para threads:
 - **User threads:** gerenciadas por *user-level libraries*, sem suporte do kernel
 - **Kernel threads:** gerenciadas diretamente pelo sistema operacional
- Deve existir uma relação entre user threads e kernel threads ...

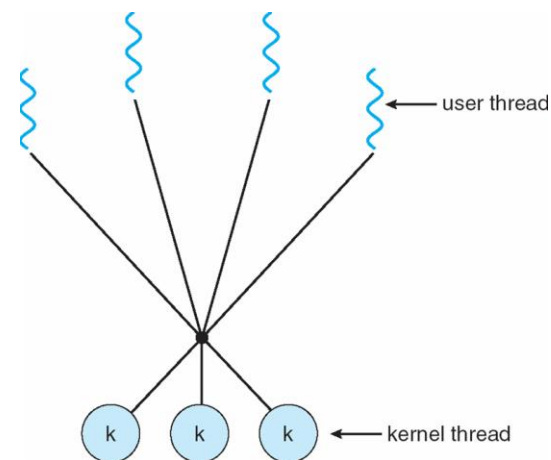
Many-to-One



One-to-One



Many-to-Many



Modelos de suporte a multithreading

- Suporte para threads:
 - **User threads:** gerenciadas por *user-level libraries*, sem suporte do kernel
 - **Kernel threads:** gerenciadas diretamente pelo sistema operacional
- Deve existir uma relação entre user threads e kernel threads ...

Many-to-One

✓ Gerenciamento de threads feito no espaço de usuário (mais eficiente)

✗ Processo inteiro irá bloquear se uma thread bloquear

✗ Múltiplas threads não irão executar em paralelo em multicore

One-to-One

✓ Maior concorrência, permite que outra thread execute se alguma fizer chamada bloqueante

✓ Permite execução paralela em múltiplos processadores

✗ Overhead: criar user-thread implica em criar kernel-thread (restringir # de kernel-threads suportado pelo sistema)

Many-to-Many

✓ Pode-se criar quantidade de user-threads que for necessária

✓ Kernel threads poderão executar em paralelo em multi-processadores

✓ Quando uma thread ficar bloqueada, kernel poderá escalonar outra thread

Programação concorrente

- Programação concorrente:
 - Duas ou mais tarefas realizadas em conjunto para efetuar um trabalho
 - Tarefa implementada como **processo** ou **thread**
- Dois aspectos da definição:
 - Cada tarefa é um “programa” sequencial
 - Programa sequencial: um único fluxo de controle (thread), portanto um programa concorrente possui múltiplos fluxos de controle
 - Trabalhar em conjunto implica:
 - Coordenação entre tarefas → sincronização
 - Troca de informações entre tarefas → comunicação

Comunicação entre tarefas

- Necessário para a produção de resultados finais consistentes
 - Programada utilizando variáveis compartilhadas ou troca de mensagens
 - Depende do paradigma de programação ...
- **Memória compartilhada**
 - Tarefas tem acesso a uma **mesma área de endereçamento**
 - Variável em memória: escrita e leitura em posições de memória
 - Comunicação **implícita**
- **Memória distribuída**
 - Tarefas possuem **áreas de endereçamento diferentes**
 - Necessário informar valores através do envio e recebimento de mensagens
 - Troca de mensagens: primitivas send e recv
 - Comunicação **explícita**

Tarefa = processo ou *thread*

Sincronização entre tarefas

- Necessário coordenar a execução de ações entre processos
 - Tanto em memória compartilhada como em memória distribuída
- Duas situações:
 - **Exclusão mútua (EM)**: duas ou mais tarefas não podem estar simultaneamente em uma **seção crítica**
 - **Condição (C)**: atrasar a execução de uma tarefa até que uma determinada condição seja verdadeira
- Exemplo: produtor-consumidor visto na última aula
 - Só pode obter dados de um buffer, se houver dados disponíveis **(C)**
 - Só escreve dados em um buffer, se houver espaço disponível **(C)**
 - Atualização e leitura consistente do objeto compartilhado **(EM)**

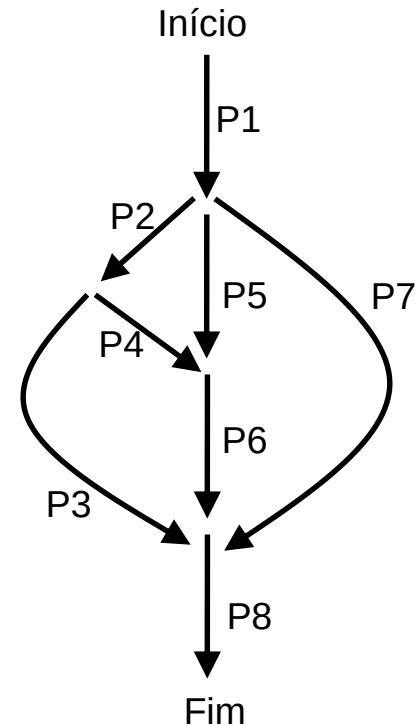
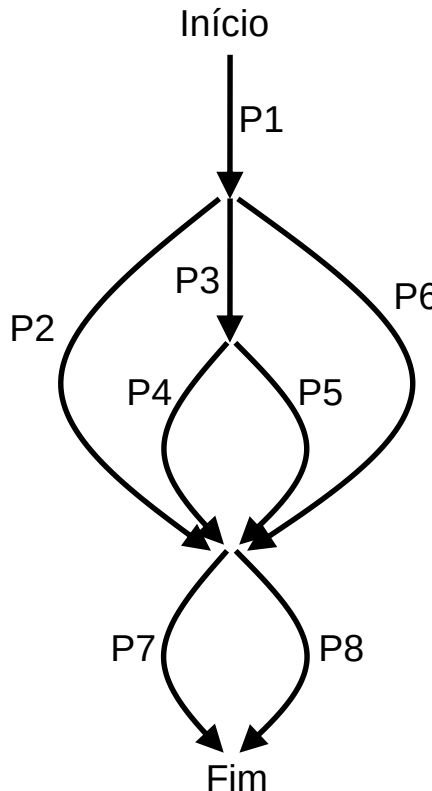
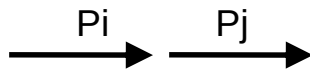
Tarefa = processo ou *thread*

Expressão da concorrência

- Forma de especificar a criação e controle de programas concorrentes
 - Grafo de precedência
 - Funções S e P
 - Operadores Cobegin/Coend
 - Fork/Join
 - Notação programação concorrente [Andrews, G.]

Grafos de precedência

- Grafos de precedência
 - Fornecer a ordem de execução de um conjunto de processos
 - Especificados através de um DAG (Directed Acyclic Graph)
 - Cada arco representa a execução de um processo
 - Um processo P_j só pode ser executado após P_i



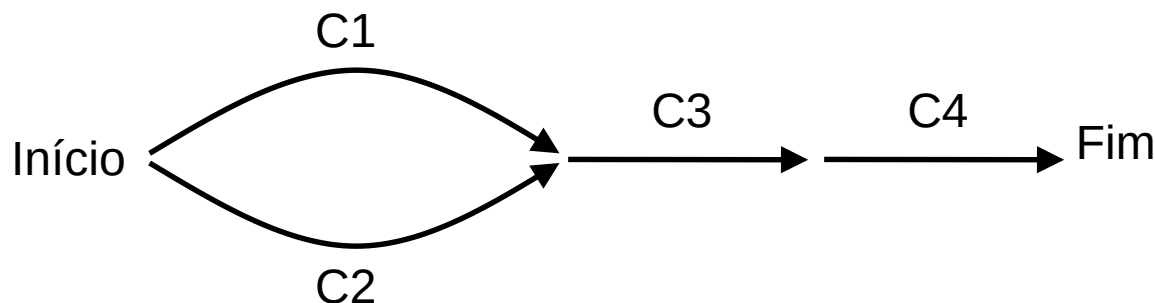
Normalmente utilizado em projeto, especificação, análise, etc

Exemplo de grafo de precedência

- Programa:

```
C1: a = x + y;  
C2: b = z + 1;  
C3: c = a - b;  
C4: w = c + 1;
```

- O comando C3 não pode ser executado antes de C1 e C2, pois C3 necessita dos novos valores das variáveis a e b. O comando C4 não pode ser executado antes que a variável c seja atualizada por C3.

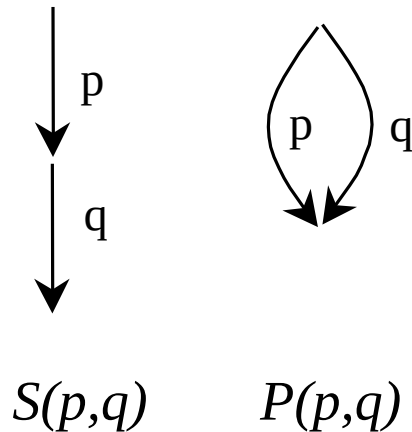


Limitações de grafo de precedência

- Não consegue representar adequadamente
 - Comunicação contínua entre duas ou mais tarefas
 - Condições usadas para criação/término de tarefas
 - Dificuldade de retratar de forma simples interações mais complexas
- Na literatura se encontram extensões ou outras propostas para contornar essas e outras limitações
 - Um dos aspectos considerados nessas extensões/propostas é fornecer capacidades para realizar prova formal de correção

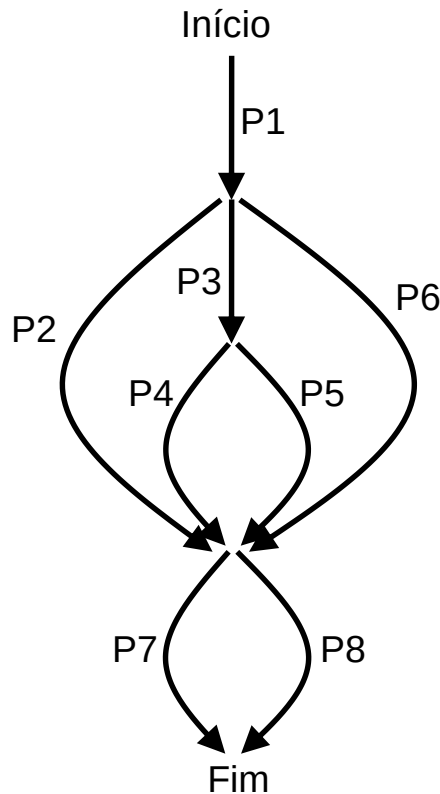
Funções S e P

- Expressam a relação de precedência entre os processos
 - $S(p, q)$: os processos p e q são executados em série
 - $P(p, q)$: os processos p e q são executados em paralelo



Grafo propriamente aninhado

- Aquele que pode ser expresso através da composição de S e P



Considera-se a figura de dentro para fora:

Passo 1: $P(p4,p5)$

Passo 2: $S(p3,P(p4,p5))$

Passo 3: $P(p2,S(p3,P(p4,p5)))$

Passo 4: $P(P(p2,S(p3,P(p4,p5))),p6)$

Passo 5: $S(p1,P(P(p2,S(p3,P(p4,p5))),p6))$

Passo 6: $P(p7,p8)$

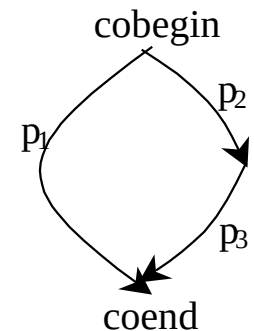
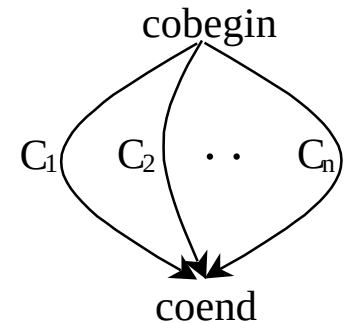
Passo 7: $S(S(p1,P(P(p2,S(p3,P(p4,p5))),p6)),P(p7,p8))$

Operadores cobegin/coend

- Propostos originalmente por Dijkstra (1965)
 - Chamados então de parbegin/parend
- Sintaxe:

cobegin C1 | C2 | ... | Cn coend

- Quando execução chega em cobegin
 - Criação de n tarefas independentes
- C_i é um trecho de código autônomo
- Quando todas as n tarefas terminam sua execução
 - Comando seguinte ao coend é executado
- Operadores “|” e “;”: equivalem a P e S, respectivamente
 - Precedência de “;” sobre |

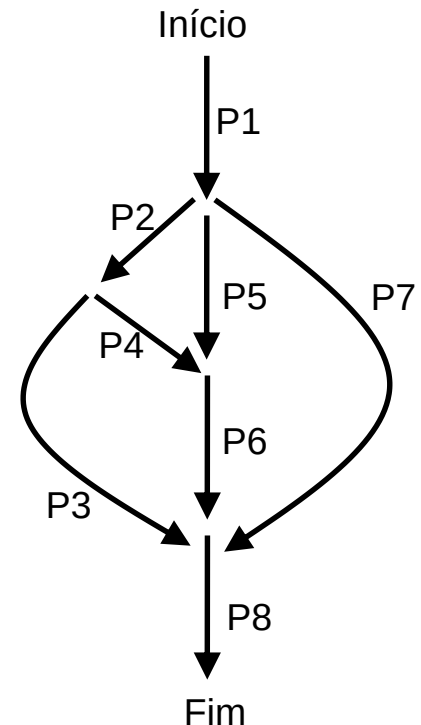


Exercício

- Represente o grafo abaixo utilizando a notação P e S (ou cobegin/coend)

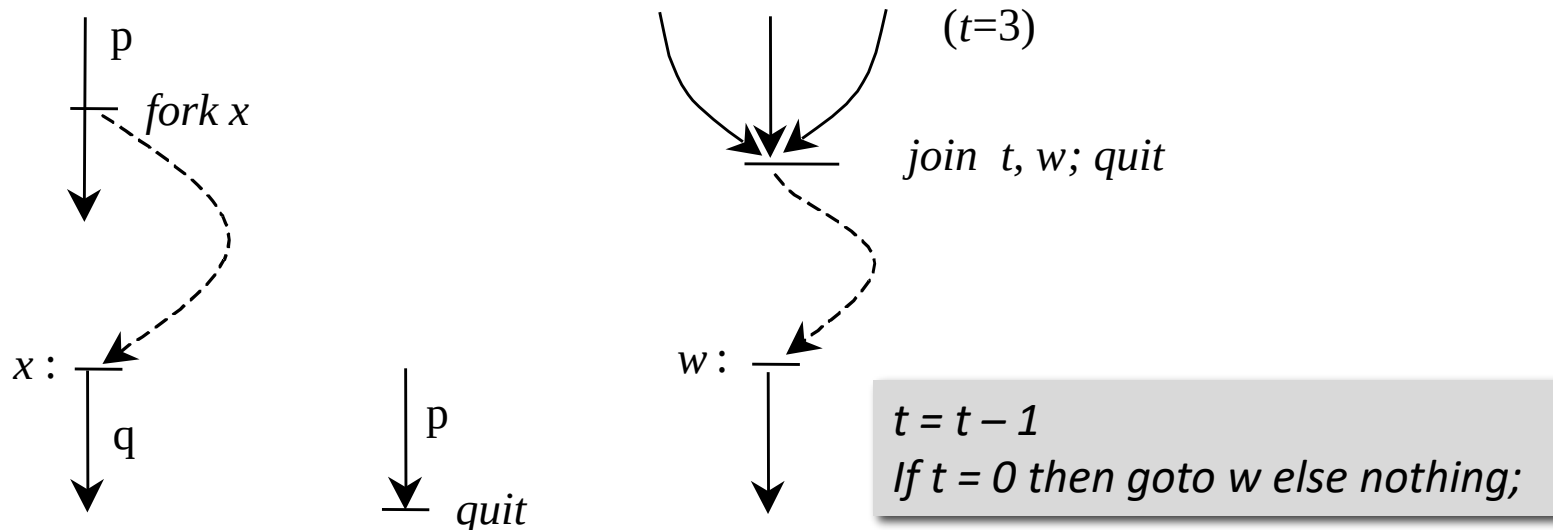
- Grafo **não é propriamente aninhado!!!**
- Não há como representar as dependências através de P e S (ou cobegin/coend)

Solução: Utilizar primitivas fork, quit e join...



Primitivas fork, join e quit

- Definidas por Conway (1963)
- Permitem descrever qualquer grafo de precedência
 - Implementadas em diversos sistemas operacionais e linguagens



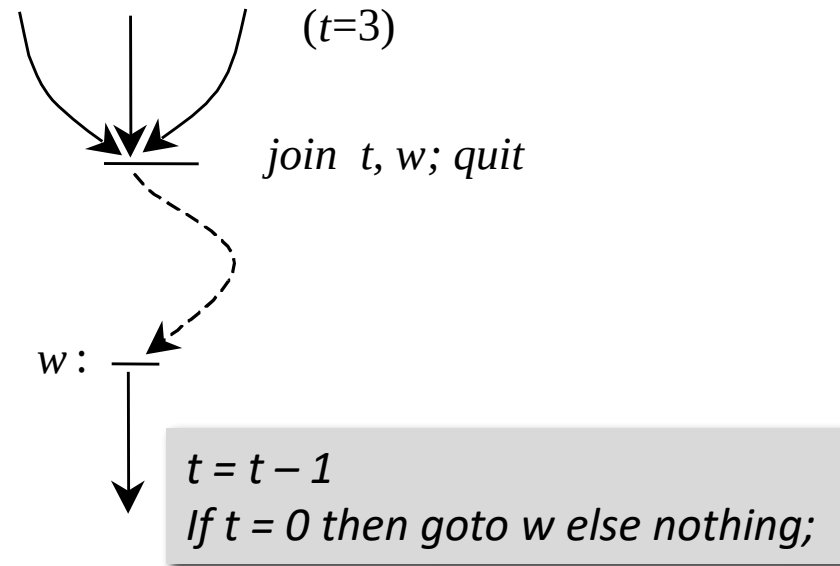
Operações *fork*, *quit* e *join*

Semântica fork()

- Produz duas execuções concorrentes em um programa
 - Uma execução inicia na instrução rotulada com x, enquanto a outra segue a instrução seguinte ao fork
- Permite a criação dinâmica de tarefas
- Área de dados (variáveis): duas alternativas possíveis
 - Pai e filho compartilham variáveis
 - Pai e filho possuem cópias privadas das variáveis
- Inconveniente:
 - Estrutura do tipo goto implícita na semântica da primitiva
 - Pouca legibilidade

Semântica join()

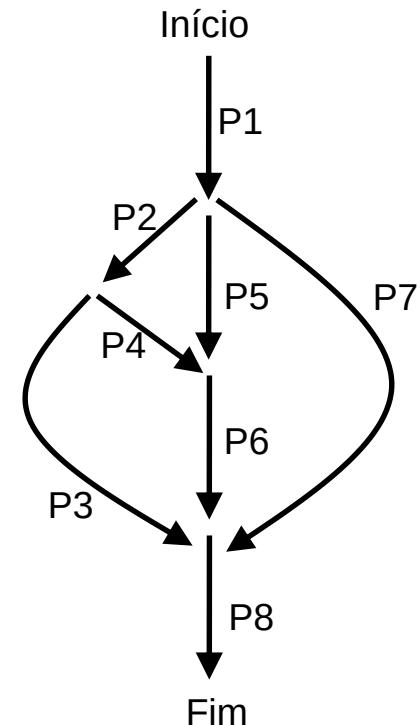
- Agrupa computações concorrentes em uma
 - Primitiva *join* é parametrizada com um contador t e um rótulo w
 - Cada processo que executa o join decrementa o contador
 - Quando contador chegar em zero, processo será direcionado para instrução rotulada em w
- Variações:
 - Agrupar n processos
 - Esperar pelo término de um processo P_k específico



Exemplo fork, join e quit

- Descrever o grafo de precedência utilizando primitivas fork, join e quit

```
t1:=2; t2:=3;  
p1; fork a2; fork a5; fork a7; quit;  
a2: p2; fork a3; fork a4; quit;  
a3: p3; join t2, a8; quit;  
a4: p4; join t1, a6; quit;  
a5: p5; join t1, a6; quit;  
a6: p6; join t2, a8; quit;  
a7: p7; join t2, a8; quit;  
a8: p8; quit;
```



Estudo de caso: fork, wait e exit no UNIX

- Semânticas diferentes: Atenção!!
- `fork( )` cria um **processo filho idêntico ao processo pai**, incluindo **cópias privativas das variáveis** do processo pai
 - Após o `fork( )` tem-se dois processos:
 - Processo pai: `fork` retorna o pid do filho
 - Processo filho: o `fork` retorna zero
 - Espaços de endereçamento distintos,
 - **Não há compartilhamento** de variáveis
 - Combinado com a chamada de sistema `exec( )`
- `wait(pid_t pid)` faz o processo que a executa esperar pelo término do processo pid
 - Equivalente ao `join`, mas com semântica diferente
- `exit(int status)` termina o processo

```
id = fork()
```

```
id = fork()
if (id == 0)
    then
        { proc. filho }
    else
        { proc. pai }
```

Notação programação concorrente (Andrews)

- Comando **co** (*concurrent*) especifica que dois ou mais comandos podem ser executados em paralelo (SR lang)
- Primeira forma:
 - Comandos são delimitados por co-oc e separados por //
 - Executa cada um dos comandos em paralelo e espera por eles para terminar
- Segunda forma:
 - Mesma semântica, apenas simplifica escrita usando quantificador
 - Executa em paralelo, n vezes, um conjunto de comandos

```
co statement 1;  
// ...  
// statement n;  
oc
```

```
co [i = 0 to n- 1] {  
    a[i] = 0; b[i] = 0;  
}
```

Multiplicação de matrizes: Notação Andrews

```
for [ i = 0 to n-1] {  
  for [ j = 0 to n-1] {  
    #compute inner product of a[i,*] and b[*],j  
    c[i,j] = 0.0 ;  
    for [k = 0 to n-1]  
      c[i,j] = c[i,j] + a[i,k]*b[k,j]  
    }  
  }  
}
```

Versão sequencial

```
co [ i = 0 to n-1] { #compute rows in parallel  
  for [ j = 0 to n-1] {  
    c[i,j] = 0.0 ;  
    for [k = 0 to n-1]  
      c[i,j] = c[i,j] + a[i,k]*b[k,j]  
    }  
  }  
}
```

Versão paralela (por linha)

Considerações finais

- Programa concorrente é mais difícil de ser implementado devido a **comunicação** e a **sincronização** entre tarefas
 - Programação distribuída e programação paralela são formas de programação concorrente
- Tarefas em programa concorrente podem ser:
 - Processos
 - Threads
- Expressão da concorrência
 - Grafo de precedência
 - Funções S e P
 - Operadores Cobegin/Coend
 - Fork/Join
 - Notação Andrews

Leituras adicionais

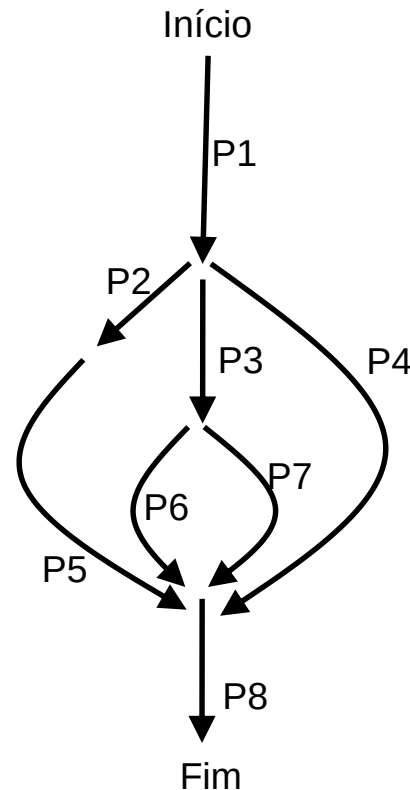
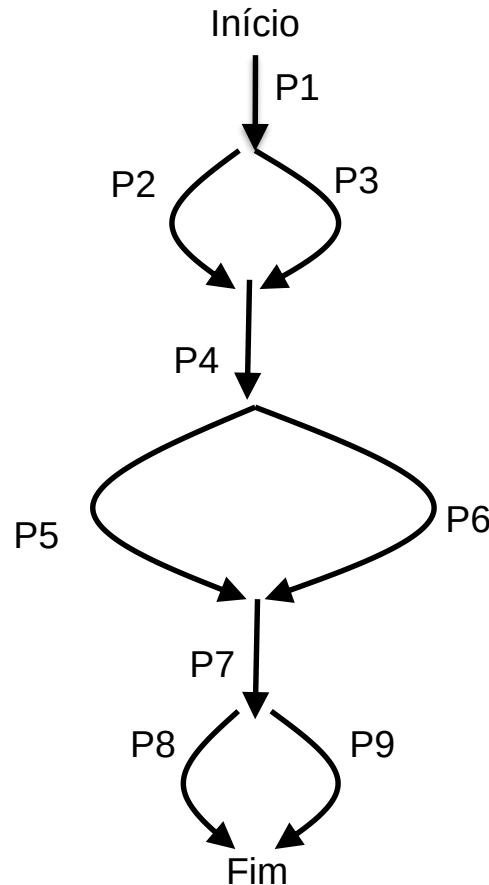
- Silberschatz, A.; Galvin, P.; Gagne, G. – *"Operating Systems Concepts"* (8th edition), John Wiley & Sons, 2009.
 - Capítulo 3: até seção 3.3 (inclusive)
 - Capítulo 4: até seção 4.2 (inclusive)
- Andrews, G. *"Foundations of Multithreaded, Parallel and Distributed Programming"*, Addison-Wesley, 2000.
 - Capítulo 1
- Toscani, S.T.; Oliveira, R.S.; Carissimi, A. *"Sistemas Operacionais e Programação Concorrente"*, Sagra-Luzzato, 2003.
 - Capítulos 2 & 3

Exercícios

1. Especifique um grafo de precedência que satisfaça as seguintes relações:
 - P2 e P3 só podem ser executados depois da execução de P1;
 - P4 só pode ser executado depois da execução de P2;
 - P5 e P6 só podem ser executados depois da execução de P4;
 - P7 só pode ser executado depois da execução de P5, P6 e P3;

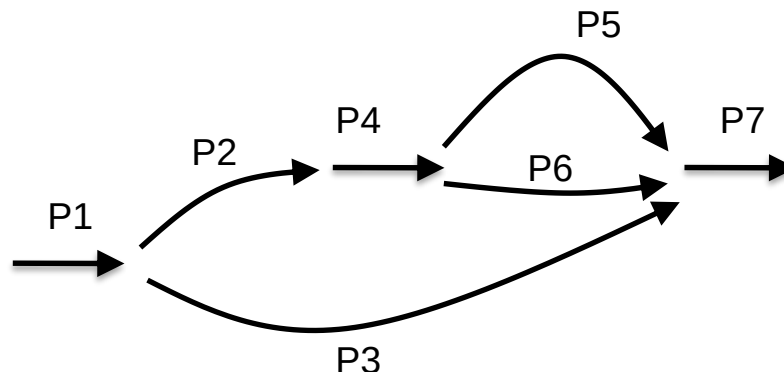
Exercícios

2. Utilizando as primitivas fork, quit e join (modelo clássico) especifique a execução dos processos de modo a satisfazer as relações de precedência



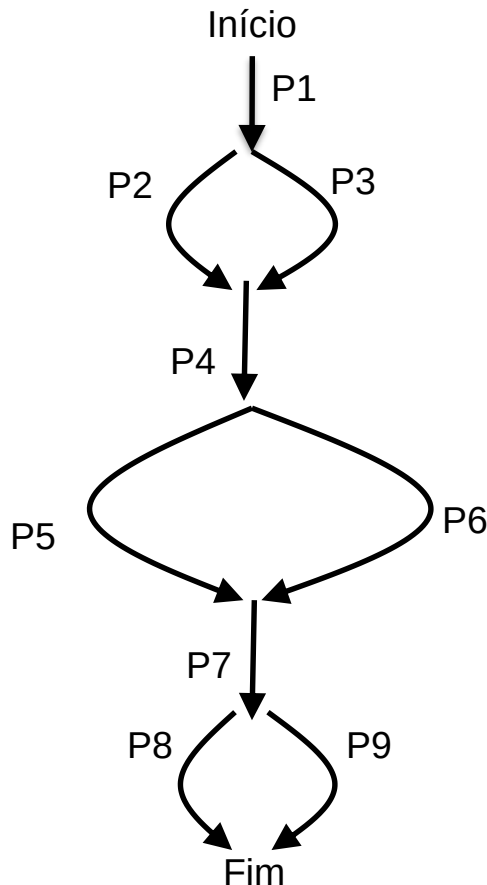
Exercícios (Respostas)

1. Especifique um grafo de precedência que satisfaça as seguintes relações:
 - P2 e P3 podem ser executados depois da execução de P1;
 - P4 pode ser executado depois da execução de P2;
 - P5 e P6 podem ser executados depois da execução de P4;
 - P7 só pode ser executado depois da execução de P5, P6 e P3;



Exercícios (Respostas)

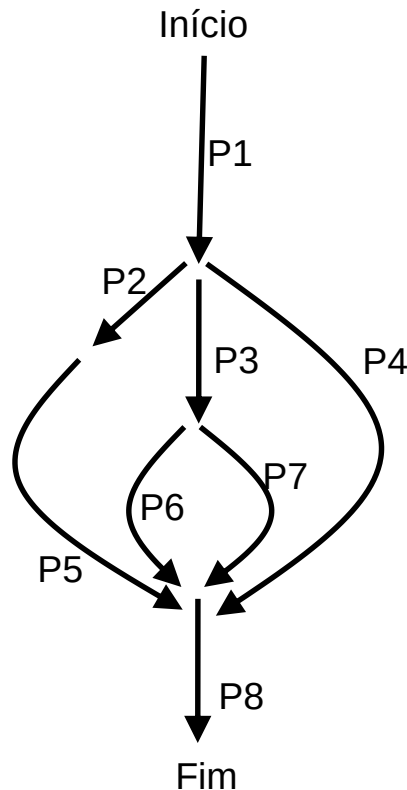
2. Utilizando as primitivas fork, quit e join (modelo clássico) especifique a execução dos processos de modo a satisfazer as relações de precedência



```
t1 := 2; t2 := 2;  
P1; fork a2; fork a3; quit;  
a2: P2; join t1, a4; quit;  
a3: P3; join t1, a4; quit;  
a4: P4; fork a5; fork a6; quit;  
a5: P5; join t2, a7; quit;  
a6: P6; join t2, a7; quit;  
a7: P7; fork a8; fork a9; quit;  
a8: P8; quit;  
a9: P9; quit;
```

Exercícios (Respostas)

2. Utilizando as primitivas fork, quit e join (modelo clássico) especifique a execução dos processos de modo a satisfazer as relações de precedência

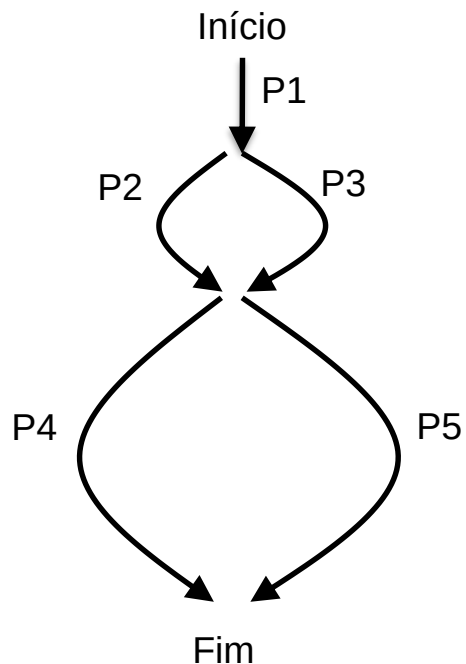


```
t1 := 4;  
P1; fork a2; fork a3; fork a4; quit;  
a2: P2; fork a5; quit;  
a3: P3; fork a6; fork a7; quit;  
a4: P4; join t1, a8; quit;  
a5: P5; join t1, a8; quit;  
a6: P6; join t1, a8; quit;  
a7: P7; join t1, a8; quit;  
a8: P8; quit;
```

Exercícios

Atenção – Fork logo após join

- O último processo a passar pelo join (quando t1 se tornará 0) será direcionado para ax, e executará os comandos fork para criação de P4 e P5



```
t1 := 2;  
P1; fork a2; fork a3; quit;  
a2: P2; join t1, ax; quit;  
a3: P3; join t1, ax; quit;  
ax: fork a4; fork a5; quit;  
a4: P4; quit;  
a5: P5; quit;
```